

Лабораторная работа № 6

Физическая реализация локальной базы данных, заполнение БД и серверная авторизация

1. Место работы в курсе

В лабораторной работе № 5 студент установил PostgreSQL, создал роль `study_app` и базу `study_space`, подключил Prisma ORM 6.19.2 и подтвердил соединение диагностическим запросом. Прикладные таблицы ещё не создавались, а данные StudySpace продолжают находиться в TypeScript-массиве.

В шестой работе логическая предметная область переводится в физическую схему PostgreSQL. Создаются таблицы пользователей, учебных мест и избранного, применяются ключи и ограничения, выполняется миграция и детерминированное начальное заполнение. После этого учебный `DemoAuthProvider` заменяется серверными маршрутами регистрации, входа, проверки текущего пользователя и выхода. Защищённая операция избранного выполняет авторизацию на сервере.

В TheraCity используются более многочисленные сущности, но StudySpace не копирует их названия или данные. Аналогия состоит в последовательности: Prisma Schema → миграция → seed → Prisma Client → серверная проверка пользователя → защищённая операция.

2. Цель работы

Спроектировать и физически создать локальную реляционную базу StudySpace в PostgreSQL, объяснить назначение и содержимое каждой таблицы, заполнить базу начальными данными через Prisma Client и реализовать учебную серверную авторизацию на Next.js с хешированием паролей, подписанной cookie-сессией и проверкой прав перед изменением избранного.

3. Задачи работы

В ходе выполнения работы студент должен:

- удалить временную модель `SetupCheck` из LP5;
- описать модели `User`, `StudyPlace` и `FavoritePlace`;
- выбрать физические типы PostgreSQL для каждого поля;
- назначить первичные, уникальные и внешние ключи;
- создать составной первичный ключ таблицы избранного;
- задать правила `onDelete` для зависимых строк;
- создать индекс для обратного поиска избранного по `placeId`;
- сформировать первую миграцию Prisma Migrate;
- изучить сгенерированный SQL до заполнения базы;
- проверить физические таблицы, столбцы и ограничения;

- написать идемпотентный seed-скрипт;
- заполнить все таблицы контролируемые учебными данными;
- создать единый экземпляр PrismaClient;
- реализовать серверную валидацию полей;
- хешировать пароль с помощью bcrypt;
- создать подписанную сессию с jose;
- сохранить сессию в cookie с безопасными атрибутами;
- реализовать регистрацию, вход, текущего пользователя и выход;
- не возвращать passwordHash клиенту;
- проверять существование пользователя в базе при авторизации;
- защитить добавление и удаление избранного;
- проверить API вручную и через данные PostgreSQL;
- выполнить prisma validate, migrate status, lint и build.

4. Теоретическая часть

4.1 Физическая модель реляционной базы

Физическая реализация переводит понятия предметной области в конкретные таблицы, столбцы, типы, ключи, ограничения и индексы PostgreSQL. Prisma Schema описывает модели приложения, а Prisma Migrate создаёт SQL, который изменяет фактическую схему базы. Официальная документация Prisma указывает, что модели соответствуют сущностям приложения, а скалярные поля моделей соответствуют столбцам реляционной базы. ([Prisma: Models](#))

В StudySpace создаются три таблицы:

- User — зарегистрированные пользователи;
- StudyPlace — учебные места, которые показываются в галерее и на карте;
- FavoritePlace — связи между пользователями и выбранными ими местами.

Типы, размеры строк и правила удаления в приложении являются проектными решениями курса. Их необходимо обосновать для собственной предметной области.

4.2 Какие данные хранит каждая таблица

User хранит:

- id — стабильный UUID пользователя;
- name — отображаемое имя;
- email — нормализованный адрес для входа, уникальный в базе;
- passwordHash — результат хеширования пароля, но не исходный пароль;
- createdAt — время создания учётной записи;
- updatedAt — время последнего изменения строки.

StudyPlace хранит:

- id — UUID учебного места;
- slug — уникальный читаемый идентификатор маршрута или данных;
- title — название;
- summary — краткое описание;
- address — адрес или описание расположения;
- image — путь к изображению из public;
- imageAlt — альтернативный текст изображения;
- latitude и longitude — координаты для карты;
- createdAt и updatedAt — временные метки.

FavoritePlace хранит:

- userId — внешний ключ на User;
- placeId — внешний ключ на StudyPlace;
- createdAt — время добавления в избранное.

Таблица FavoritePlace является явной таблицей связи многие-ко-многим. Составной первичный ключ (userId, placeId) не позволяет одному пользователю добавить одно место дважды.

4.3 Первичные, уникальные и внешние ключи

Документация PostgreSQL определяет первичный ключ как уникальный и не допускающий NULL идентификатор строки. Добавление первичного ключа также создаёт уникальный B-tree индекс. Уникальное ограничение обеспечивает уникальность значения или комбинации значений. Внешний ключ требует, чтобы значение ссылалось на существующую строку другой таблицы, и тем самым поддерживает ссылочную целостность. ([PostgreSQL: Constraints](#))

В Prisma используются:

```
// Назначает одиночный первичный ключ.
id String @id @default(uuid()) @db.Uuid

// Требуется уникальный email.
email String @unique

// Назначает составной первичный ключ таблицы связи.
@@id([userId, placeId])
```

Уникальная проверка в интерфейсе не заменяет ограничение базы: два параллельных запроса могут пройти предварительную проверку. Окончательная гарантия уникальности принадлежит базе данных.

4.4 Связи Prisma и внешние ключи

Prisma описывает связь полями отношений и скалярным полем внешнего ключа. Поле `relation` существует на уровне Prisma Client, а `userId` или `placeId` существует как физический столбец. ([Prisma: Relations](#))

```
// Связывает строку избранного с существующим пользователем.  
user User @relation(fields: [userId], references: [id], onDelete: Cascade)
```

PostgreSQL определяет CASCADE как автоматическое удаление зависимых строк при удалении родительской строки. Для FavoritePlace это уместно: связь избранного не имеет смысла без пользователя или места. Само удаление пользователя не должно удалять независимые учебные места.

4.5 Выбор типов PostgreSQL

PostgreSQL предоставляет числовые, строковые, логические, временные, UUID и другие типы. Prisma позволяет дополнить скалярный тип нативным типом базы через `@db`. ([PostgreSQL: Data Types](#); [Prisma Schema Reference](#))

В работе используются:

- `@db.Uuid` для идентификаторов;
- `@db.VarChar(n)` для строк с осмысленной максимальной длиной;
- `@db.Text` для описания переменной длины;
- `@db.Decimal(9, 6)` для координат с фиксированной точностью;
- `@db.Timestampt(3)` для времени с часовым поясом.

`Decimal` перед отправкой в JSON преобразуется в `number` или строку, поскольку объект Prisma `Decimal` не является обычным JSON-числом. Для координат лабораторного диапазона `Number(place.latitude)` достаточно; в финансовых расчётах такое преобразование может быть недопустимо.

4.6 Индексы

Индекс помогает находить строки без полного просмотра таблицы, но занимает место и добавляет работу при изменении данных. PostgreSQL автоматически создаёт индексы для первичных и уникальных ограничений, однако внешний ключ на ссылающейся стороне не получает индекс автоматически. Документация PostgreSQL отмечает, что индексировать такие столбцы часто полезно. ([PostgreSQL: Constraints](#); [Indexes](#))

Составной первичный ключ FavoritePlace начинается с `userId` и хорошо поддерживает запрос «избранное пользователя». Дополнительный `@@index([placeId])` нужен для обратного запроса «какие пользователи выбрали место» и для операций, связанных с удалением или анализом места.

4.7 Миграции Prisma Migrate

Миграция — версионизируемое описание изменения физической схемы. В `development` команда `prisma migrate dev` сравнивает модель, историю миграций и базу, создаёт SQL-миграцию, применяет её и обновляет таблицу `_prisma_migrations`. Документация

запрещает использовать migrate dev как production-команду; там применяют migrate deploy. ([Prisma: Development and production](#))

```
# Создаём и применяем первую migration в локальной development-базе.  
npx prisma migrate dev --name init_study_space
```

Файл migration.sql является частью исходного кода. Его необходимо прочитать и сохранить. Ручное редактирование уже применённой миграции создаёт расхождение истории.

4.8 Начальное заполнение базы

Seeding создаёт данные, необходимые для запуска или проверки приложения. Официальная документация Prisma рекомендует seed для воспроизводимого заполнения development-базы. В Prisma ORM 6 команда задаётся в поле prisma.seed package.json и запускается через prisma db seed; после migrate reset seed также может выполняться автоматически. ([Prisma: Seeding](#))

Идемпотентный seed можно запускать повторно без создания дубликатов. Для уникальных полей используется upsert: существующая строка обновляется, отсутствующая создаётся. В StudySpace seed создаёт одного демонстрационного пользователя, три учебных места и одну строку избранного.

4.9 CRUD и Prisma Client

CRUD означает Create, Read, Update и Delete. Prisma Client предоставляет методы create, findUnique, findMany, update, delete, upsert и групповые операции. ([Prisma: CRUD](#))

Серверные маршруты используют:

- user.create для регистрации;
- user.findUnique для входа и проверки сессии;
- favoritePlace.upsert для идемпотентного добавления;
- favoritePlace.deleteMany для безопасного повторного удаления;
- studyPlace.findMany в LP7 для выдачи галереи.

В select следует перечислять только безопасные поля. Возврат всей строки User может случайно отправить passwordHash клиенту.

4.10 Хеширование паролей

Пароль нельзя хранить в исходном виде. Официальное руководство Next.js по авторизации показывает хеширование пароля перед записью пользователя и подчёркивает сложность собственной auth-реализации. В учебном проекте используется bcrypt с cost factor 12. ([Next.js: Authentication](#))

```
// Преобразуем пароль в необратимый хеш перед записью в User.  
const passwordHash = await hash(password, 12);
```

bcrypt включает соль в результат. При входе compare проверяет пароль относительно сохранённого хеша. Клиенту возвращается только общее сообщение «Неверный email или пароль», чтобы не раскрывать существование адреса.

4.11 Аутентификация, сессия и авторизация

Руководство Next.js разделяет три понятия:

- аутентификация проверяет личность пользователя;
- управление сессией сохраняет состояние входа между запросами;
- авторизация решает, какие данные и действия доступны пользователю.

В лабораторной работе регистрация и вход выполняют аутентификацию. Подписанный JWT в cookie хранит минимальный идентификатор сессии. Перед изменением избранного сервер проверяет подпись токена и существование пользователя в базе — это авторизация защищённой операции.

Скрытие кнопки на клиенте не является авторизацией. Злоумышленник может отправить HTTP-запрос напрямую, поэтому проверка выполняется внутри Route Handler рядом с изменением данных.

4.12 Подписанная cookie-сессия

Официальная документация Next.js описывает stateless-сессию как подписанные или зашифрованные данные в cookie и рекомендует использовать библиотеку управления сессией, например jose. В payload следует хранить минимальные данные, а не пароль или лишнюю персональную информацию. ([Next.js: Authentication](#))

В StudySpace JWT содержит userId и name, подписывается алгоритмом HS256 секретом AUTH_SECRET и действует семь дней. Секрет хранится только на сервере.

Cookie задаётся с атрибутами:

- httpOnly — клиентский JavaScript не читает cookie;
- secure — cookie передаётся только по HTTPS в production;
- sameSite=lax — ограничивает часть межсайтовых отправок;
- path=/ — cookie доступна маршрутам приложения;
- maxAge — задаёт срок действия.

Функция cookies в современных версиях Next.js асинхронна. Читать cookie можно в серверном коде, а устанавливать или удалять — в Server Function или Route Handler. ([Next.js: cookies](#))

4.13 Ограничения учебной auth-реализации

Официальное руководство Next.js рекомендует для production использовать проверенную auth-библиотеку, поскольку готовое решение должно учитывать дополнительные угрозы и сценарии: восстановление доступа, подтверждение email, смену пароля, отзыв сессий,

ограничение частоты запросов, аудит, CSRF, MFA и безопасное управление ключами.
([Next.js: Authentication](#))

Реализация лабораторной работы предназначена для понимания потока данных и не объявляется полноценной production-системой. Stateless JWT нельзя отозвать до истечения срока без дополнительного механизма. Для более сложного проекта можно использовать database sessions или специализированную библиотеку.

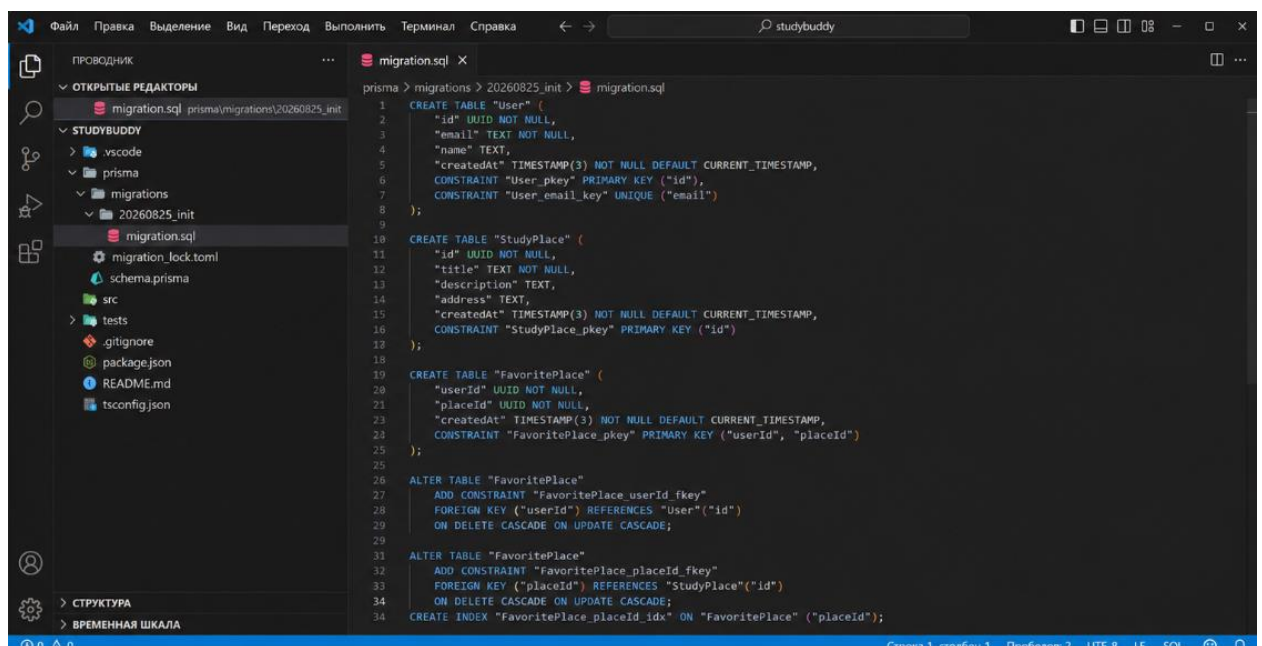
5. Практическое задание

Этап 1. Проверка исходного состояния

Проверьте результат ЛР5:

- `psql` подключается к `study_space` под `study_app`;
- `npx prisma validate` выполняется;
- `scripts/check-prisma.ts` выводит правильную базу и роль;
- в базе нет предметных таблиц;
- `.env` не отслеживается `Git`;
- `npm run lint` и `npm run build` проходят.

Сделайте резервную копию или отдельную ветку.



```
1 CREATE TABLE "User" (  
2   "id" UUID NOT NULL,  
3   "email" TEXT NOT NULL,  
4   "name" TEXT,  
5   "createdAt" TIMESTAMPT(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,  
6   CONSTRAINT "User_pkey" PRIMARY KEY ("id"),  
7   CONSTRAINT "User_email_key" UNIQUE ("email")  
8 );  
9  
10 CREATE TABLE "StudyPlace" (  
11   "id" UUID NOT NULL,  
12   "title" TEXT NOT NULL,  
13   "description" TEXT,  
14   "address" TEXT,  
15   "createdAt" TIMESTAMPT(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,  
16   CONSTRAINT "StudyPlace_pkey" PRIMARY KEY ("id")  
17 );  
18  
19 CREATE TABLE "FavoritePlace" (  
20   "userId" UUID NOT NULL,  
21   "placeId" UUID NOT NULL,  
22   "createdAt" TIMESTAMPT(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,  
23   CONSTRAINT "FavoritePlace_pkey" PRIMARY KEY ("userId", "placeId")  
24 );  
25  
26 ALTER TABLE "FavoritePlace"  
27   ADD CONSTRAINT "FavoritePlace_userId_fkey"  
28   FOREIGN KEY ("userId") REFERENCES "User"("id")  
29   ON DELETE CASCADE ON UPDATE CASCADE;  
30  
31 ALTER TABLE "FavoritePlace"  
32   ADD CONSTRAINT "FavoritePlace_placeId_fkey"  
33   FOREIGN KEY ("placeId") REFERENCES "StudyPlace"("id")  
34   ON DELETE CASCADE ON UPDATE CASCADE;  
35  
36 CREATE INDEX "FavoritePlace_placeId_idx" ON "FavoritePlace" ("placeId");
```

Этап 2. Установка библиотек авторизации

Выполните:

```
# Устанавливаем хеширование паролей и подпись JWT.  
npm install bcrypt jose  
npm install -D @types/bcrypt
```

Сгенерируйте секрет локально:

```
# Создаём криптографически случайный секрет и копируем вывод в .env.
node -e
"console.log(require('node:crypto').randomBytes(32).toString('base64url'))"
```

Добавьте AUTH_SECRET в .env и заполнение в .env.example. Не помещайте фактический секрет в отчёт.

Этап 3. Проектирование таблиц

Удалите SetupCheck и внесите модели User, StudyPlace и FavoritePlace из приложения. Для каждого поля в отчёте укажите:

- прикладное назначение;
- тип Prisma;
- нативный тип PostgreSQL;
- допускается ли NULL;
- значение по умолчанию;
- ключ или ограничение;
- пример безопасного значения.

Не копируйте названия сущностей TheraCity. Если ранее была выбрана другая предметная область, переименуйте модели и поля согласованно.

Этап 4. Создание и изучение миграции

Выполните:

```
# Форматируем и проверяем новую модель данных.
npx prisma format
npx prisma validate

# Создаём и применяем первую физическую схему.
npx prisma migrate dev --name init_study_space
```

Откройте созданный prisma/migrations/.../migration.sql. Найдите CREATE TABLE, PRIMARY KEY, UNIQUE, FOREIGN KEY, ON DELETE CASCADE и CREATE INDEX. Не изменяйте применённый файл.

Этап 5. Проверка физической схемы

Выполните `npx prisma migrate status`, затем подключитесь через `psql`:

```
-- Показываем прикладные таблицы схемы public.
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

-- Показываем физические столбцы и типы таблицы User.
SELECT column_name, data_type, is_nullable
```

```
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name = 'User'
ORDER BY ordinal_position;
```

Дополнительно используйте \d "User", \d "StudyPlace" и \d "FavoritePlace". Двойные кавычки необходимы для имён с заглавными буквами.

Этап 6. Создание seed-скрипта

Добавьте prisma/seed.ts и настройку prisma.seed в package.json. Seed должен:

- хешировать демонстрационный пароль;
- выполнять upsert пользователя по email;
- выполнять upsert каждого места по slug;
- добавлять одну связь FavoritePlace через составной ключ;
- не создавать дубликаты при повторном запуске;
- отключать PrismaClient в finally.

Данные и фотографии должны соответствовать собственному StudySpace из ЛР3.

Этап 7. Заполнение и проверка данных

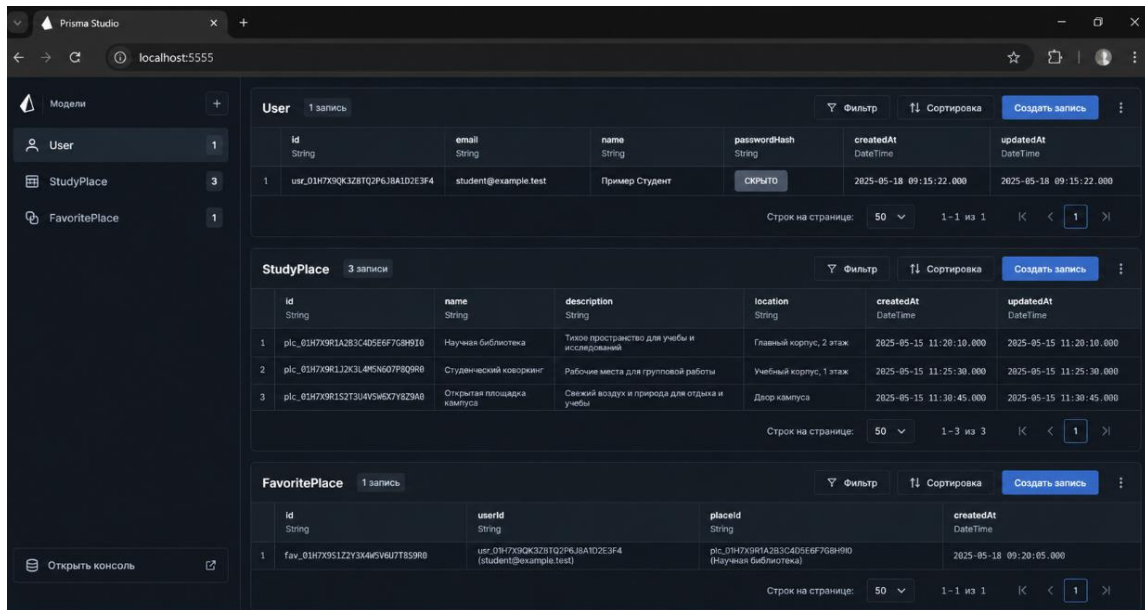
Выполните дважды:

```
# Повторный запуск не должен увеличивать число строк.
npx prisma db seed
npx prisma db seed
```

Проверьте количество строк через psql:

```
-- Подтверждаем заполнение каждой прикладной таблицы.
SELECT 'User' AS table_name, count(*) FROM "User"
UNION ALL
SELECT 'StudyPlace', count(*) FROM "StudyPlace"
UNION ALL
SELECT 'FavoritePlace', count(*) FROM "FavoritePlace";
```

Откройте npx prisma studio и визуально проверьте связи. Не изменяйте passwordHash вручную.



Этап 8. Серверная валидация и PrismaClient

Сохраните единый `app/lib/prisma.ts` из ЛР5. Создайте `app/lib/validation.ts`. Валидация должна выполняться на сервере даже при наличии атрибутов `required` и `minLength` в HTML.

Проверьте:

- `name` после `trim` содержит 2–80 символов;
- `email` нормализуется `trim` и `lower-case`;
- `email` соответствует базовой форме адреса;
- пароль содержит не менее 8 символов;
- `confirmPassword` совпадает;
- API не принимает произвольные типы как готовые данные.

Этап 9. Реализация сессии

Создайте `app/lib/auth.ts`. Реализуйте:

- `signSession` — подписывает минимальный `payload`;
- `createSession` — устанавливает `httpOnly` cookie;
- `readSession` — проверяет подпись и срок;
- `deleteSession` — удаляет cookie;
- отсутствие вывода токена и `AUTH_SECRET` в Console.

Создайте `app/lib/current-user.ts`, который после проверки токена запрашивает пользователя в базе и возвращает только `id`, `name` и `email`.

Этап 10. Регистрация

Создайте POST /api/auth/register. Маршрут должен:

- прочитать JSON;
- проверить поля на сервере;
- нормализовать email;
- хешировать пароль;
- создать User;
- обработать конфликт уникального email кодом 409;
- создать сессию;
- вернуть безопасный объект user и статус 201;
- не вернуть passwordHash.

Этап 11. Вход, текущий пользователь и выход

Создайте:

- POST /api/auth/login;
- GET /api/auth/me;
- POST /api/auth/logout.

Login должен использовать одинаковую ошибку для отсутствующего пользователя и неверного пароля. Me возвращает user: null для гостя и безопасные поля для вошедшего пользователя. Logout удаляет cookie.

The screenshot shows a web application interface on the left and a network log on the right. The web application, titled 'Учебный API', has four sections: 'Регистрация' (Registration), 'Вход' (Login), 'Профиль' (Profile), and 'Выход' (Logout). The network log shows a series of requests and responses. The first request is a POST to /api/auth/register with a status of 201. The second request is a POST to /api/auth/login with a status of 200. The third request is a GET to /api/auth/me with a status of 200. The fourth request is a POST to /api/auth/logout with a status of 200. The response for the login request is a JSON object with fields 'id', 'name', and 'email'.

Имя	Статус	Тип	Инициатор	X	Заголовки	Тело	Предварительный просмотр	Ответ	Индикатор	Время
POST register /api/auth/register	201	fetch	client.js:42	1				{		
POST login /api/auth/login	200	fetch	client.js:58	2				"id": "a3f7c2d1-9b8e-4a6a-bb1c-8d9e2f7a6c11",		
GET me /api/auth/me	200	fetch	client.js:74	3				"name": "Алексей Петров",		
POST logout /api/auth/logout	200	fetch	client.js:90	4				"email": "alexey.petrov@example.test"		
				5				}		
				6						

Запросов: 4 Передано: 1,26 кБ Получено: 1,26 кБ

Этап 12. Защищённая операция избранного

Создайте POST и DELETE /api/favorites/[placeId]. Оба метода сначала вызывают getCurrentUser. Для гостя возвращается 401. POST проверяет существование места и выполняет upsert связи. DELETE использует user.id из сессии, а не идентификатор из тела запроса.

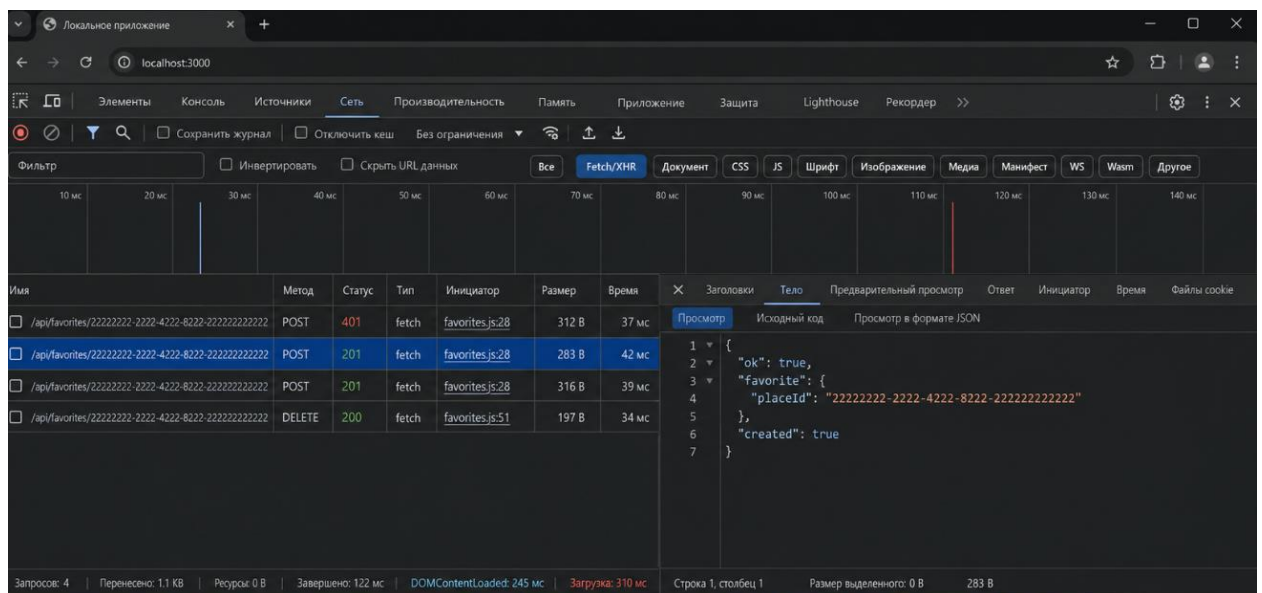
Тем самым пользователь может изменять только собственную связь FavoritePlace.

Этап 13. Ручная проверка API

Запустите npm run dev и используйте формы LP2 или вкладку Network DevTools. Проверьте:

- регистрация нового email → 201;
- повторная регистрация → 409;
- выход → 200;
- неверный пароль → 401;
- правильный вход → 200 и Set-Cookie;
- GET /api/auth/me после входа → объект user;
- POST избранного гостем → 401;
- POST избранного после входа → 201;
- повторный POST не создаёт дубликат;
- DELETE удаляет только связь текущего пользователя.

Не вставляйте пароль или JWT в отчёт. Значение cookie на скриншоте закройте.



Этап 14. Итоговая проверка

Выполните:

```
# Проверяем схему, миграцию, генерацию, код и production-сборку.
npx prisma format
npx prisma validate
```

```
npx prisma migrate status
npx prisma generate
npm run lint
npm run build
```

Для отчёта нужны собственные скриншоты: migration.sql; npx prisma migrate status; \d для трёх таблиц; результаты count(*) после двух seed-запусков; три таблицы в Prisma Studio; успешная регистрация, вход, me, выход и защищённое избранное во вкладке Network. Скройте пароли, AUTH_SECRET, DATABASE_URL, JWT и значение cookie.

Учебная auth-реализация показывает базовый поток и не считается готовой production-системой. В отчёте отдельно перечислите отсутствующие механизмы: подтверждение email, восстановление пароля, отзыв токенов, ограничение частоты, CSRF-защита, MFA и аудит входов.

6. Требования к результату

После выполнения работы студент должен предоставить проект, в котором:

- SetupCheck удалена до первой миграции;
- созданы модели и физические таблицы User, StudyPlace и FavoritePlace;
- назначены типы, ключи, уникальные ограничения и внешние ключи;
- составной ключ предотвращает дубликаты избранного;
- зависимые связи удаляются по выбранному правилу Cascade;
- миграция сохранена в репозитории и имеет понятное имя;
- migrate status сообщает, что схема актуальна;
- seed можно запускать повторно без увеличения количества строк;
- каждая таблица заполнена и объяснена в отчёте;
- пароль хранится только как bcrypt-хеш;
- AUTH_SECRET и DATABASE_URL не опубликованы;
- cookie имеет httpOnly, sameSite, path, maxAge и production-условие secure;
- register, login, me и logout возвращают согласованные статусы;
- passwordHash не попадает в JSON;
- защищённое избранное проверяет пользователя на сервере;
- гостевой запрос изменения данных получает 401;
- npm run lint и npm run build выполняются без ошибок.

7. Содержание отчёта

Отчёт должен содержать:

- название и цель работы;
- схему связей User → FavoritePlace ← StudyPlace;
- описание назначения каждой таблицы;
- описание каждого столбца с типом, NULL и ограничениями;
- обоснование составного первичного ключа;

- обоснование onDelete: Cascade;
- schema.prisma;
- migration.sql;
- скриншоты \d трёх таблиц;
- вывод migrate status;
- seed.ts с комментариями;
- результаты двух seed-запусков и count(*);
- скриншоты таблиц Prisma Studio;
- validation.ts;
- auth.ts и current-user.ts;
- код четырёх auth-маршрутов;
- код защищённого маршрута избранного;
- матрицу проверенных HTTP-сценариев со статусами;
- доказательство отсутствия passwordHash в ответах;
- результаты lint и build;
- перечень ограничений учебной авторизации;
- ответы на контрольные вопросы;
- вывод по работе.

Контрольные вопросы:

- Что входит в физическую схему реляционной базы?
- Чем модель Prisma отличается от физической таблицы?
- Для чего нужен первичный ключ?
- Чем уникальное ограничение отличается от обычного индекса?
- Что гарантирует внешний ключ?
- Почему FavoritePlace имеет составной первичный ключ?
- Что делает onDelete: Cascade?
- Почему внешний ключ не всегда получает индекс автоматически?
- Для чего нужен migration.sql?
- Чем migrate dev отличается от migrate status?
- Почему нельзя редактировать уже применённую миграцию без понимания последствий?
- Что означает идемпотентный seed?
- Чем upsert отличается от create?
- Почему пароль нельзя хранить в исходном виде?
- Что хранит bcrypt-хеш?
- Чем аутентификация отличается от авторизации?
- Что такое stateless session?
- Зачем JWT подписывается секретом?
- Какие данные допустимо хранить в payload сессии?
- Для чего нужны httpOnly, secure и sameSite?
- Почему скрытая кнопка не является авторизацией?

- Почему current user повторно проверяется в базе?
- Почему passwordHash нельзя возвращать клиенту?
- Зачем для неверного email и неверного пароля используется одно сообщение?
- Какие ограничения имеет учебная реализация?

8. Источники

Теоретическая часть подготовлена по официальной документации PostgreSQL, Prisma ORM и Next.js. Состав таблиц StudySpace, ограничения длины, cost factor 12, срок сессии семь дней, демонстрационные данные и матрица проверок являются проектными решениями курса.

- PostgreSQL: Data Types — <https://www.postgresql.org/docs/current/datatype.html>
- PostgreSQL: Constraints — <https://www.postgresql.org/docs/current/ddl-constraints.html>
- PostgreSQL: Indexes — <https://www.postgresql.org/docs/current/indexes.html>
- PostgreSQL: Transactions — <https://www.postgresql.org/docs/current/tutorial-transactions.html>
- Prisma: Models — <https://www.prisma.io/docs/orm/v6/prisma-schema/data-model/models>
- Prisma: Relations — <https://www.prisma.io/docs/orm/v6/prisma-schema/data-model/relations>
- Prisma Schema Reference — <https://www.prisma.io/docs/orm/v6/reference/prisma-schema-reference>
- Prisma: Development and production — <https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/development-and-production>
- Prisma: Seeding — <https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/seeding>
- Prisma: CRUD — <https://www.prisma.io/docs/orm/v6/prisma-client/queries/crud>
- Prisma: Transactions and batch queries — <https://www.prisma.io/docs/orm/v6/prisma-client/queries/transactions>
- Prisma: Handling exceptions and errors — <https://www.prisma.io/docs/orm/v6/prisma-client/debugging-and-troubleshooting/handling-exceptions-and-errors>
- Prisma: Next.js troubleshooting — <https://www.prisma.io/docs/orm/v6/more/troubleshooting/nextjs>
- Next.js: Authentication — <https://nextjs.org/docs/app/guides/authentication>
- Next.js: cookies — <https://nextjs.org/docs/app/api-reference/functions/cookies>
- Next.js: Route Handlers — <https://nextjs.org/docs/app/getting-started/route-handlers>
- Next.js: Environment Variables — <https://nextjs.org/docs/app/guides/environment-variables>

1. Структура новых и изменённых файлов

Показывает серверные файлы базы данных и авторизации.

```
study-space/
├── app/
│   ├── api/
│   │   ├── auth/
│   │   │   ├── login/route.ts
│   │   │   ├── logout/route.ts
│   │   │   ├── me/route.ts
│   │   │   └── register/route.ts
│   │   └── favorites/
│   │       └── [placeId]/route.ts
│   └── lib/
│       ├── auth.ts
│       ├── current-user.ts
│       ├── prisma.ts
│       └── validation.ts
├── prisma/
│   ├── migrations/
│   │   └── ..._init_study_space/migration.sql
│   ├── schema.prisma
│   └── seed.ts
├── .env
├── .env.example
└── package.json
```

2. Файл prisma/schema.prisma

```
// Генерирует Prisma Client для серверного TypeScript-кода.
generator client {
  provider = "prisma-client-js"
}

// Подключает PostgreSQL через локальную переменную DATABASE_URL.
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Хранит зарегистрированного пользователя и его безопасные метаданные.
model User {
  id          String    @id @default(uuid()) @db.Uuid
  name        String    @db.VarChar(80)
  email       String    @unique @db.VarChar(254)
  passwordHash String    @db.VarChar(255)
  createdAt   DateTime  @default(now()) @db.Timestamptz(3)
  updatedAt   DateTime  @updatedAt @db.Timestamptz(3)

  // Связывает пользователя со строками его избранного.
  favorites FavoritePlace[]
}

// Хранит учебное место, отображаемое в галерее и на карте.
model StudyPlace {
  id          String    @id @default(uuid()) @db.Uuid
  slug        String    @unique @db.VarChar(80)
  title       String    @db.VarChar(160)
  summary     String    @db.Text
```

```

address String @db.VarChar(240)
image String @db.VarChar(240)
imageAlt String @db.VarChar(240)
latitude Decimal @db.Decimal(9, 6)
longitude Decimal @db.Decimal(9, 6)
createdAt DateTime @default(now()) @db.Timestamptz(3)
updatedAt DateTime @updatedAt @db.Timestamptz(3)

// Связывает место со всеми пользовательскими добавлениями в избранное.
favorites FavoritePlace[]
}

// Реализует явную связь многие-ко-многим между User и StudyPlace.
model FavoritePlace {
  userId String @db.Uuid
  placeId String @db.Uuid
  createdAt DateTime @default(now()) @db.Timestamptz(3)

  // Удаляем зависимую связь вместе с пользователем.
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Удаляем зависимую связь вместе с учебным местом.
  place StudyPlace @relation(fields: [placeId], references: [id], onDelete: Cascade)

  // Запрещаем повтор одной пары пользователь-место.
  @@id([userId, placeId])

  // Ускоряем обратный поиск связей по учебному месту.
  @@index([placeId])
}

```

3. Файл prisma/seed.ts

```

// Идемпотентно заполняет локальную development-базу учебными данными.

import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { hash } from "bcrypt";

// Для отдельного seed-процесса создаём собственный клиент.
const prisma = new PrismaClient();

// Описываем те же три места, которые использовались в клиентской ЛР3.
const places = [
  {
    slug: "library",
    title: "Научная библиотека",
    summary: "Тихое место для индивидуальной работы с книгами и конспектами.",
    address: "Университетская улица, 1",
    image: "/images/places/library.jpg",
    imageAlt: "Читальный зал научной библиотеки",
    latitude: "55.754400",
    longitude: "37.619200",
  },
  {
    slug: "coworking",
    title: "Студенческий коворкинг",
    summary: "Пространство для групповых проектов и обсуждений.",
    address: "Учебный корпус, аудитория 210",
    image: "/images/places/coworking.jpg",
    imageAlt: "Столы для групповой работы в студенческом коворкинге",
    latitude: "55.758300",
    longitude: "37.610600",
  },
  {
    slug: "campus",

```

```

    title: "Открытая площадка кампуса",
    summary: "Место для чтения и коротких учебных встреч на свежем воздухе.",
    address: "Центральный двор кампуса",
    image: "/images/places/campus.jpg",
    imageAlt: "Открытая площадка с местами для работы в кампусе",
    latitude: "55.749800",
    longitude: "37.630200",
  },
];

async function main() {
  // Хешируем только демонстрационный пароль локальной development-базы.
  const passwordHash = await hash("StudyPass_2026!", 12);

  // Создаём или обновляем одного демонстрационного пользователя.
  const user = await prisma.user.upsert({
    where: { email: "student@example.test" },
    update: {
      name: "Демонстрационный студент",
      passwordHash,
    },
    create: {
      name: "Демонстрационный студент",
      email: "student@example.test",
      passwordHash,
    },
  });

  // Создаём или обновляем каждое место по уникальному slug.
  for (const place of places) {
    await prisma.studyPlace.upsert({
      where: { slug: place.slug },
      update: place,
      create: place,
    });
  }

  // Находим библиотеку, чтобы создать демонстрационную связь избранного.
  const library = await prisma.studyPlace.findUniqueOrThrow({
    where: { slug: "library" },
  });

  // Upsert составного ключа не создаёт дубликат при повторном seed.
  await prisma.favoritePlace.upsert({
    where: {
      userId_placeId: {
        userId: user.id,
        placeId: library.id,
      },
    },
    update: {},
    create: {
      userId: user.id,
      placeId: library.id,
    },
  });

  // Выводим проверяемый итог без пароля и хеша.
  console.log("Seed complete: 1 user, 3 places, 1 favorite");
}

main()
  .catch((error) => {
    console.error("Seed failed:", error);
    process.exitCode = 1;
  })
  .finally(async () => {
    // Закрываем соединения отдельного seed-процесса.
  });
}

```

```
    await prisma.$disconnect();
  });
```

4. Файл app/lib/prisma.ts

```
// Переиспользует PrismaClient между горячими перезагрузками Next.js.

import { PrismaClient } from "@prisma/client";

// Добавляем типизированное поле клиента в globalThis.
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

// Используем существующий клиент или создаём его один раз.
export const prisma = globalForPrisma.prisma ?? new PrismaClient();

// Сохраняем клиент глобально только в development.
if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

5. Файл app/lib/validation.ts

```
// Нормализует и проверяет непроверенные данные auth-запросов.

export type RegistrationInput = {
  name: string;
  email: string;
  password: string;
  confirmPassword: string;
};

// Приводим email к форме, используемой уникальным полем базы.
export function normalizeEmail(value: unknown): string {
  return String(value ?? "").trim().toLowerCase();
}

// Проверяем поля регистрации и возвращаем данные только при успехе.
export function validateRegistration(value: unknown): {
  data: RegistrationInput | null;
  errors: string[];
} {
  const source =
    typeof value === "object" && value !== null
      ? (value as Record<string, unknown>)
      : {};

  const data: RegistrationInput = {
    name: String(source.name ?? "").trim(),
    email: normalizeEmail(source.email),
    password: String(source.password ?? ""),
    confirmPassword: String(source.confirmPassword ?? ""),
  };

  const errors: string[] = [];

  // Проверяем учебные ограничения длины отображаемого имени.
  if (data.name.length < 2 || data.name.length > 80) {
    errors.push("Имя должно содержать от 2 до 80 символов");
  }

  // Выполняем базовую синтаксическую проверку email.
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(data.email)) {
```

```

    errors.push("Некорректный email");
  }

  // Требуем минимальную длину учебного пароля.
  if (data.password.length < 8) {
    errors.push("Пароль должен содержать не менее 8 символов");
  }

  // Исключаем случайную регистрацию с несовпадающим подтверждением.
  if (data.password !== data.confirmPassword) {
    errors.push("Пароли не совпадают");
  }

  return {
    data: errors.length === 0 ? data : null,
    errors,
  };
}

```

6. Файл app/lib/auth.ts

```

// Подписывает, проверяет, устанавливает и удаляет учебную cookie-сессию.

import { SignJWT, jwtVerify } from "jose";
import { cookies } from "next/headers";

const COOKIE_NAME = "study_session";
const SESSION_MAX_AGE = 60 * 60 * 24 * 7;

// Читаем серверный секрет и сразу запрещаем запуск с пустым значением.
function getSecret(): Uint8Array {
  const value = process.env.AUTH_SECRET;
  if (!value) {
    throw new Error("AUTH_SECRET is not configured");
  }
  return new TextEncoder().encode(value);
}

// Храним в сессии только минимальные данные пользователя.
export type SessionPayload = {
  userId: string;
  name: string;
};

// Создаём подписанный JWT с ограниченным сроком действия.
async function signSession(payload: SessionPayload): Promise<string> {
  return new SignJWT(payload)
    .setProtectedHeader({ alg: "HS256" })
    .setIssuedAt()
    .setExpirationTime("7d")
    .sign(getSecret());
}

// Устанавливаем cookie только из серверного Route Handler.
export async function createSession(payload: SessionPayload): Promise<void> {
  const token = await signSession(payload);
  const store = await cookies();

  store.set(COOKIE_NAME, token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
    path: "/",
    maxAge: SESSION_MAX_AGE,
  });
}

```

```

// Возвращаем payload только после успешной проверки подписи и срока.
export async function readSession(): Promise<SessionPayload | null> {
  const token = (await cookies()).get(COOKIE_NAME)?.value;
  if (!token) return null;

  try {
    const { payload } = await jwtVerify(token, getSecret(), {
      algorithms: ["HS256"],
    });

    if (typeof payload.userId !== "string" || typeof payload.name !== "string") {
      return null;
    }

    return { userId: payload.userId, name: payload.name };
  } catch {
    // Любая ошибка подписи, формата или срока превращает запрос в гостевой.
    return null;
  }
}

// Удаляем cookie при выходе пользователя.
export async function deleteSession(): Promise<void> {
  (await cookies()).delete(COOKIE_NAME);
}

```

7. Файл app/lib/current-user.ts

```

// Проверяет сессию и загружает актуального пользователя из базы.

import { prisma } from "../prisma";
import { readSession } from "../auth";

export async function getCurrentUser() {
  // Сначала проверяем подписанный идентификатор сессии.
  const session = await readSession();
  if (!session) return null;

  // Затем убеждаемся, что пользователь всё ещё существует в базе.
  return prisma.user.findUnique({
    where: { id: session.userId },
    select: {
      id: true,
      name: true,
      email: true,
    },
  });
}

```

8. Файл app/api/auth/register/route.ts

```

// Регистрирует пользователя, хеширует пароль и создаёт cookie-сессию.

import { Prisma } from "@prisma/client";
import { hash } from "bcrypt";
import { NextResponse } from "next/server";
import { createSession } from "@app/lib/auth";
import { prisma } from "@app/lib/prisma";
import { validateRegistration } from "@app/lib/validation";

export const runtime = "nodejs";

export async function POST(request: Request) {

```

```

try {
  // Читаем непроверенное тело запроса.
  const body = await request.json();
  const validation = validateRegistration(body);

  // Возвращаем ошибки клиента без обращения к базе.
  if (!validation.data) {
    return NextResponse.json(
      { errors: validation.errors },
      { status: 400 },
    );
  }

  const { name, email, password } = validation.data;

  // Хешируем пароль до записи строки User.
  const passwordHash = await hash(password, 12);

  // Создаём пользователя и сразу выбираем только безопасные поля.
  const user = await prisma.user.create({
    data: { name, email, passwordHash },
    select: { id: true, name: true, email: true },
  });

  // Сохраняем минимальные данные в подписанной cookie-сессии.
  await createSession({ userId: user.id, name: user.name });

  return NextResponse.json({ user }, { status: 201 });
} catch (error) {
  // Преобразуем конфликт уникального email в согласованный HTTP 409.
  if (
    error instanceof Prisma.PrismaClientKnownRequestError &&
    error.code === "P2002"
  ) {
    return NextResponse.json(
      { errors: ["Этот email уже зарегистрирован"] },
      { status: 409 },
    );
  }

  // Не отправляем клиенту внутренний объект ошибки или секреты.
  return NextResponse.json(
    { errors: ["Внутренняя ошибка сервера"] },
    { status: 500 },
  );
}
}

```

9. Файл app/api/auth/login/route.ts

```

// Проверяет email и пароль, затем создаёт сессию пользователя.

import { compare } from "bcrypt";
import { NextResponse } from "next/server";
import { createSession } from "@app/lib/auth";
import { prisma } from "@app/lib/prisma";
import { normalizeEmail } from "@app/lib/validation";

export const runtime = "nodejs";

export async function POST(request: Request) {
  try {
    // Нормализуем email и приводим пароль к строке.
    const body = (await request.json()) as Record<string, unknown>;
    const email = normalizeEmail(body.email);
    const password = String(body.password ?? "");

```

```

    if (!email || !password) {
      return NextResponse.json(
        { errors: ["Заполните email и пароль"] },
        { status: 400 },
      );
    }

    // Для проверки пароля временно загружаем passwordHash только на сервере.
    const account = await prisma.user.findUnique({ where: { email } });

    // Используем одно сообщение для отсутствующего аккаунта и неверного пароля.
    if (!account || !(await compare(password, account.passwordHash))) {
      return NextResponse.json(
        { errors: ["Неверный email или пароль"] },
        { status: 401 },
      );
    }

    // Создаём подписанную сессию после успешной проверки.
    await createSession({ userId: account.id, name: account.name });

    // Явно формируем DTO без passwordHash.
    const user = {
      id: account.id,
      name: account.name,
      email: account.email,
    };

    return NextResponse.json({ user });
  } catch {
    return NextResponse.json(
      { errors: ["Внутренняя ошибка сервера"] },
      { status: 500 },
    );
  }
}

```

10. Файл `app/api/auth/me/route.ts`

```

// Возвращает безопасные данные текущего пользователя или null для гостя.

import { NextResponse } from "next/server";
import { getCurrentUser } from "@app/lib/current-user";

export async function GET() {
  const user = await getCurrentUser();
  return NextResponse.json({ user });
}

```

11. Файл `app/api/auth/logout/route.ts`

```

// Завершает stateless-сессию удалением cookie в браузере.

import { NextResponse } from "next/server";
import { deleteSession } from "@app/lib/auth";

export async function POST() {
  await deleteSession();
  return NextResponse.json({ ok: true });
}

```

12. Файл app/api/favorites/[placeId]/route.ts

```
// Защищает добавление и удаление избранного серверной проверкой пользователя.

import { NextResponse } from "next/server";
import { getCurrentUser } from "@/app/lib/current-user";
import { prisma } from "@/app/lib/prisma";

type RouteParams = {
  params: Promise<{ placeId: string }>;
};

export async function POST(_request: Request, context: RouteParams) {
  // Авторизуем действие рядом с изменением данных.
  const user = await getCurrentUser();
  if (!user) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { placeId } = await context.params;

  // Не создаём связь с отсутствующим учебным местом.
  const place = await prisma.studyPlace.findUnique({
    where: { id: placeId },
    select: { id: true },
  });
  if (!place) {
    return NextResponse.json({ error: "Place not found" }, { status: 404 });
  }

  // Upsert делает повторный POST идемпотентным для одной пары ключей.
  const favorite = await prisma.favoritePlace.upsert({
    where: { userId_placeId: { userId: user.id, placeId } },
    update: {},
    create: { userId: user.id, placeId },
  });

  return NextResponse.json({ favorite }, { status: 201 });
}

export async function DELETE(_request: Request, context: RouteParams) {
  // Гость не может изменять строки FavoritePlace.
  const user = await getCurrentUser();
  if (!user) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  const { placeId } = await context.params;

  // Удаляем только связь пользователя, полученного из проверенной сессии.
  const result = await prisma.favoritePlace.deleteMany({
    where: { userId: user.id, placeId },
  });

  return NextResponse.json({ deleted: result.count });
}
```

13. Файлы окружения

```
# .env.example документирует ключи без реальных секретов.
DATABASE_URL="postgresql://study_app:CHANGE_ME@localhost:5432/study_space?schema=public"
AUTH_SECRET="GENERATE_A_RANDOM_32_BYTE_SECRET"
```

14. Фрагменты package.json

```
{
  "scripts": {
    "db:migrate": "prisma migrate dev",
    "db:seed": "prisma db seed",
    "db:status": "prisma migrate status",
    "db:studio": "prisma studio"
  },
  "prisma": {
    "seed": "tsx prisma/seed.ts"
  }
}
```

JSON не допускает комментарии. Назначение команд описано рядом с фрагментом и в этапах работы.

15. Команды создания и проверки

```
# Устанавливаем библиотеки auth-части.
npm install bcrypt jose
npm install -D @types/bcrypt

# Создаём секрет и вручную переносим его в локальный .env.
node -e "console.log(require('node:crypto').randomBytes(32).toString('base64url'))"

# Создаём физическую схему и генерируем клиент.
npx prisma format
npx prisma validate
npx prisma migrate dev --name init_study_space
npx prisma generate

# Заполняем и визуально проверяем development-базу.
npx prisma db seed
npx prisma db seed
npx prisma studio

# Проверяем актуальность миграций и Next.js-проект.
npx prisma migrate status
npm run lint
npm run build
```

16. Что студент должен заменить

- модели и поля — на собственную предметную область, если она отличается от StudySpace;
- seed-объекты — на собственные тексты, фотографии, alt и координаты;
- демонстрационный email и пароль — на безопасные локальные тестовые значения;
- AUTH_SECRET — на случайно сгенерированный секрет;
- имена форм и маршрутов — на фактическую структуру своего проекта;
- все снимки и HTTP-результаты — на собственные.

Нельзя публиковать реальный DATABASE_URL, AUTH_SECRET, пароль, JWT, cookie, passwordHash или данные чужого проекта.